CS2103/T Week 2 Lecture Notes

Topics: SDLC process models, Git/GitHub, IDEs, and automated testing

Source: CS2103/T Week 2 Topics PDF, exported 21 Aug 2026. These notes reorganize and explain the source material in lecturer style for study.

How to Study Week 2

Big picture. Week 2 moves from simply writing code to managing software work. The core question is: how do we control change when the product, team, and uncertainty grow?

- First, understand process models: they explain how a team moves through requirements, design, coding, testing, release, and later maintenance.
- Second, treat Git and GitHub as a safety net and history reader, not just as commands to memorize.
- Third, use an IDE to reduce mechanical coding effort and to observe program behavior.
- Fourth, start testing early because every new feature can accidentally break existing behavior.

W2.1 SDLC Process Models: Basics

Why Process Models Exist

A very small software project can sometimes survive with code-and-fix: start coding, run the program, fix whatever breaks, and repeat. This has almost no planning overhead, so it can work for a short one-person program.

But as the program or team grows, code-and-fix becomes weak. The team cannot easily tell how far along the project is, divide work cleanly, or remember why earlier choices were made. Changes become more expensive because there is no shared roadmap.

The software development lifecycle, or SDLC, names the main activities software usually passes through: requirements, analysis, design, implementation, testing, deployment, operation, and maintenance. Process models describe different ways to organize those activities.

Lecturer explanation. Think of SDLC as the list of jobs that must happen; a process model is the route you choose through those jobs.

Common SDLC Activities

Activity	Purpose	Typical output
Requirements	Find out what the system should do and what constraints it must satisfy.	Requirement statements, user needs, acceptance criteria.
Analysis	Understand the problem, domain, and feasibility more deeply.	Problem model, assumptions, risks, clarified scope.
Design	Decide how the system will be structured before implementation.	Architecture, component design, UI/data/API design.
Implementation	Write and integrate the code.	Working code, builds, commits.
Testing	Run the system under specified conditions and compare with expected behavior.	Test results, bug reports, confidence evidence.
Deployment	Make the software available to users.	Release package, production setup, delivery process.
Operation and maintenance	Keep the software useful after release as users, defects, and environments change.	Bug fixes, improvements, future work items.

Sequential / Waterfall Model

The sequential model, often called the waterfall model, treats development as a linear movement through stages. A stage is completed, produces artifacts, and those artifacts become input for the next stage.

- Example: requirements are gathered first, then used by the design stage.
- In a strict sequential model, the project moves forward only; once a stage is finished, the team does not revise it later.

- In practice, teams often relax this and allow a later stage to send work back to the previous stage, but that means redoing work considered finished.

Strength	Why it helps
Easy stage-level tracking	You can say the project is in requirements, design, implementation, or testing.
Clear artifacts	Each stage has a visible output for the next stage.
Suitable for stable problems	If requirements are well understood and unlikely to change, estimates can be more reliable.

Weakness	Why it hurts
Poor fit for unclear problems	Users may not know their true needs until they see or use a working system.
Late feedback	Integration and user contact may happen near the end, when changes cost more.
Hidden stage overrun	Progress within a long stage can still be hard to see until the delay has already grown.

Iterative Model

The iterative model builds the software through several cycles. Each iteration can pass through requirements, design, implementation, testing, and even deployment. It produces a new product version or usable improvement.

Feedback from one iteration is used to improve later iterations. If an implementation task took longer than expected, later estimates can be adjusted. If a feature is not well received by users, it can be changed or removed.

Key distinction. A sequential project is divided by activity. An iterative project is divided by bounded cycles, each designed to produce evidence and learning.

Breadth-First and Depth-First Iterations

Approach	Meaning	Minesweeper example
Breadth-first	Each iteration evolves most major components and feature areas in parallel, producing a working product each time.	An early version is playable but primitive: text UI, fixed board size, and limited mine layouts.
Depth-first	Each iteration focuses deeply on one component, feature area, risk, or assumption; early iterations may not produce a complete working product.	One iteration builds the full UI without game logic; another focuses only on minefield generation.
Mixed	A project can combine both styles depending on the risk and learning goal of each iteration.	One iteration may improve the playable game while also exploring a risky algorithm.

Iterating and incrementing are related but different. To iterate is to rework existing material using feedback. To increment is to add something new. Most real projects are both iterative and incremental.

The result of an iteration is an increment: a usable improvement or addition to the product, not merely a new code version.

Evidence rule. Before an iteration starts, decide what success evidence should exist at the end: a passing test, a working demo, a target-user response, or a decision made from the result.

Risk-Driven Iterations and the Spiral Idea

An early iteration is valuable because it lets the team discover wrong assumptions while changing direction is still cheap. Therefore, risky assumptions should be tested early: unfamiliar technology, unclear user need, or a difficult performance target.

Ordering iterations by risk is the central idea of the spiral model. The team spirals through planning, building, and learning while reducing the biggest risks first.

AI Impact on Process Models

AI coding tools make candidate implementations cheaper and faster. However, deciding what to build and verifying that the result is correct remain difficult.

A vague requirement that might once have produced a clarifying question from a teammate can now produce a fast but wrong implementation. This makes precise specification and verification even more important.

Sequential vs Iterative: Lecturer Comparison

Question	Sequential answer	Iterative answer
How is work divided?	By activity: requirements, design, implementation, testing.	By cycles: each iteration produces learning or an increment.
When does feedback arrive?	Often late, especially user feedback.	Earlier and repeatedly.
Best fit?	Stable, well-understood requirements.	Uncertain requirements, changing needs, risky assumptions.
Main danger?	Discovering mistakes when revising is expensive.	Running iterations without clear evidence or decisions.

W2.2 and W2.3 RCS: Git, GitHub, and Revision History

AI's Impact on Git and GitHub

The Week 2 notes frame Git as something developers increasingly lean on rather than manually type from memory. AI can run commands for you, but you must still understand what those commands do.

- Remembering exact commands matters less than understanding their effect.
- Being able to go back matters more because AI can change many files quickly.
- Committing before risky work gives you a safe return point.
- Reading diffs and pull requests matters more because you may review code you did not personally write.

- Small commits with clear messages are easier to review, understand, and undo.

Week 2 Git-Mastery Tours

Item	Tour focus	What you should be able to explain
W2.2a	Tour 2: Backing up a repo on the cloud.	Why a remote copy protects work and enables sharing.
W2.2b	Tour 3: Working off a remote repo.	How local and remote repositories relate during fetch/pull/push style workflows.
W2.3a	Tour 4: Using revision history.	How history helps inspect old states, understand changes, and trace decisions.
W2.3b	Tour 5: Fine-tuning revision history.	Why history can be tidied or shaped so it stays readable and useful.

W2.4 IDEs: Basic Features

An Integrated Development Environment, or IDE, supports many development activities in one tool. It is not just a text editor; it brings editing, building, running, debugging, testing, and other support together.

IDE part	What it does	Why it matters for beginners
Source code editor	Syntax coloring, auto-completion, code navigation, error highlighting, and snippets.	Helps you notice mistakes earlier and move around code faster.
Compiler/interpreter and build support	Compiles, links, runs, and packages programs.	Reduces friction when repeatedly running your project.

Debugger	Runs the program step by step so you can inspect runtime behavior.	Lets you see what the code actually does, not what you hoped it would do.
Additional tools	Testing support, UI builders, version-control support, runtime simulation, modeling, AI assistance, and collaboration.	Turns the IDE into a broader engineering workspace.

Examples of IDEs

- Java: Eclipse, IntelliJ IDEA, NetBeans.
- C#, C++: Visual Studio.
- Swift: Xcode.
- Python: PyCharm.
- Multiple languages: VS Code.
- Some experienced developers, especially those with UNIX backgrounds, prefer lightweight but powerful editors such as Vim or NeoVim.

The source also points students to setup guides for IntelliJ IDEA and VS Code. In this course, the practical goal is to know enough IDE usage to work productively on the individual project.

W2.5 Introduction to Automated Testing

What Testing Means

The source uses the IEEE idea of testing: operate a system or component under specified conditions, observe or record the results, and evaluate some aspect of the system.

A test case specifies how to perform a test. At minimum, it gives the input to the software under test, or SUT, and the expected behavior.

> **Lecturer explanation.** A test case is a promise written before or during testing: if we do this input, the system should behave like that. Testing is the act of checking that promise.

Test Case Example: Browser

Part	Details
Input	Start the browser using a blank page with the vertical scrollbar disabled. Then load longfile.html from the test data folder.
Expected behavior	The scrollbar should be automatically enabled after longfile.html loads.
Failure	If the scrollbar remains disabled, the test case fails.
Possible defect	The underlying bug might be something like an uninitialized variable, although the test case itself could also be wrong.

How to Execute a Test Case

1. Feed the specified input to the SUT.
2. Observe the actual output or behavior.
3. Compare the actual output with the expected output.

A test case failure is a mismatch between expected behavior and actual behavior. It indicates a potential defect. The word potential matters because the test itself may contain an incorrect expectation.

Regression Testing

A regression is an unintended and undesirable effect caused by modifying a system. Regression testing means re-testing the software to detect such regressions.

The usual method is to retest all related components, even those already tested before. Regression testing is more effective when done frequently after small changes. Manual regression testing can become too expensive, so automation makes it practical.

Concept	Meaning	Practical lesson
Regression	A new change breaks existing behavior.	Do not assume old features still work after adding a new feature.
Regression testing	Retesting to detect regressions.	Run tests repeatedly, especially after small changes.
Automation	Tests are executed and judged programmatically.	Makes frequent testing affordable and precise.

Automated Testing

An automated test case can be run programmatically, and the pass/fail result is also determined programmatically. Compared with manual testing, automation reduces repeated effort and improves precision because manual testing is prone to human error.

The optional Week 2 item mentions automated testing of CLI applications. For this course project, that connects naturally to running command-line inputs and checking exact output.

AI's Impact on Testing

- AI reduces the effort needed to draft test cases, write test code, and set up testing mechanisms.
- Because automation is easier, not having time to write tests is a weaker excuse.
- AI cannot know expected behavior unless a human or specification defines it.
- If the same AI writes both implementation and tests, it may repeat the same misunderstanding in both.
- Automated regression tests become more valuable because AI agents can make large changes quickly.

Connections to Your iP Work

Week 2 idea	How it appears in your project
Iterative development	You completed Level 0 through Level 6 as small increments rather than building everything at once.
Git history	Each level and extension can be tagged, committed, reviewed, and pushed.
IDE support	IntelliJ or another IDE helps navigate classes such as Baby, Task, Todo, Deadline, Event, and Command.
Regression testing	Your UI test plan reruns old behavior after each new feature such as mark, unmark, deadline, event, delete, and enum refactoring.
Automation	The test-ui script runs command-line scenarios and compares exact expected output.

Exam and Tutorial Style Questions

- Explain why code-and-fix may work for a tiny one-person program but not for a larger team project.
- Compare sequential and iterative process models using feedback timing, suitability, and risk.
- Use Minesweeper to explain breadth-first and depth-first iterations.
- Define an increment and explain why a code version alone may not be a useful increment.
- Explain why AI coding makes specification and verification more important, not less important.
- Explain why small commits with clear messages are useful when using AI-generated changes.
- Name the main parts of an IDE and explain how each helps development.
- Define test case, SUT, failure, regression, regression testing, and automated test case.
- Explain why a failing test indicates a potential defect rather than guaranteed implementation bug.
- Explain why automated regression testing is especially valuable after small frequent changes.

One-Page Recap

- SDLC activities include requirements, analysis, design, implementation, testing, deployment, operation, and maintenance.
- Sequential models are linear and easy to track at stage level, but weak when requirements are uncertain or feedback arrives late.
- Iterative models build through cycles, use feedback, and can be breadth-first, depth-first, or mixed.
- Good iterations end in evidence that supports a decision.
- Risky assumptions should be tested early; this is the spirit of the spiral model.
- AI makes coding faster but increases the need for precise requirements, careful verification, readable Git history, and automated regression tests.
- IDEs integrate editing, building, running, debugging, testing, and other development support.
- Testing compares actual behavior with expected behavior; regression testing checks that changes did not break existing behavior.
- Automated tests reduce repeated effort and human error, but expected behavior still has to come from a trustworthy specification or human understanding.

Source Coverage Map

- **W2.1a:** SDLC introduction, code-and-fix, lifecycle activities, maintenance feedback.
- **W2.1b:** Sequential/waterfall model, artifacts, strengths, and weaknesses.
- **W2.1c:** Iterative model, increments, breadth-first/depth-first, risk, spiral model, AI impact.
- **W2.2a-b:** GitHub remote backup and working from a remote repo as Git-Mastery tours.
- **W2.3a-b:** Using and fine-tuning revision history as Git-Mastery tours.
- **W2.4a-b:** IDE definition, parts, examples, and setup references.
- **W2.5a-d:** Testing definition, test cases, regression testing, automation, CLI testing item, AI impact.